

Notebook guiado: cómo funciona nuestra aplicación Shiny

Lectura paso a paso de Laboratorio de Portafolios

Sebastián Egaña Santibáñez (segana@fen.uchile.cl)

Nicolás Leiva Díaz (nleivad@fen.uchile.cl)

2026-10-05

Propósito

Este notebook explica, parte por parte, el código de la aplicación ubicada en shiny_portafolio/app.R.

La idea central es:

```
datos → funciones financieras → inputs → reactivos → outputs
```

Para ejecutar la aplicación:

```
shiny::runApp("clases_ia/shiny_portafolio")
```

1. Estructura de Shiny

Toda aplicación Shiny tiene una interfaz y un servidor:

```
ui <- fluidPage(  
  # Lo que verá el usuario  
)  
  
server <- function(input, output, session) {  
  # Cálculos y respuestas  
}  
  
shinyApp(ui = ui, server = server)
```

La interfaz define controles y espacios de salida. El servidor recibe acciones, calcula resultados y actualiza la pantalla. La aplicación queda esperando eventos del usuario.

2. Paquetes y datos

```
library(shiny)  
library(readr)  
library(ggplot2)
```

Paquete	Responsabilidad
shiny	Interfaz, eventos y reactividad
readr	Lectura del CSV
ggplot2	Gráfico de riesgo y retorno

La aplicación busca el CSV en varias rutas:

```
rutas_datos <- c(  
  "data/portafolio_retornos.csv",  
  "../data/portafolio_retornos.csv",
```

```

"clases_ia/data/portafolio_retornos.csv"
)

ruta_datos <- rutas_datos[file.exists(rutas_datos)][1]

if (is.na(ruta_datos)) {
  stop("No se encontró el archivo de datos.")
}

datos <- read_csv(ruta_datos, show_col_types = FALSE)

```

Esto permite iniciar la app desde la carpeta de la aplicación o desde la raíz del repositorio.

3. Preparar los activos

```

activos <- c("SPY", "EFA", "IJS", "EEM", "AGG")
retornos <- as.matrix(datos[activos])
retornos_medios <- colMeans(retornos)
covarianzas <- cov(retornos)

```

Seleccionamos las columnas, convertimos los retornos en una matriz y calculamos los insumos financieros. La covarianza es importante porque el riesgo del portafolio depende también de la diversificación.

4. Normalizar pesos

Los sliders trabajan entre 0 y 100, pero el modelo necesita pesos que sumen 1:

```

normalizar_pesos <- function(pesos) {
  if (sum(pesos) == 0) {
    return(rep(1 / length(pesos), length(pesos)))
  }

  pesos / sum(pesos)
}

```

La condición inicial evita dividir por cero cuando todos los sliders están en cero. En ese caso usamos un portafolio equiponderado.

5. Calcular las métricas financieras

```

calcular_metricas <- function(pesos, retornos_medios, covarianzas,
                             tasa_libre = 0.02) {
  retorno <- sum(pesos * retornos_medios) * 12
  varianza <- as.numeric(t(pesos) %*% covarianzas %*% pesos)
  volatilidad <- sqrt(varianza) * sqrt(12)
  sharpe <- (retorno - tasa_libre) / volatilidad

  list(
    retorno = retorno,
    volatilidad = volatilidad,
    sharpe = sharpe
  )
}

```

La función calcula:

```

retorno = suma(peso por retorno)
varianza = t(pesos) %*% covarianzas %*% pesos
Sharpe = (retorno - tasa libre de riesgo) / volatilidad

```

Los factores 12 y sqrt(12) anualizan datos mensuales.

6. Simular portafolios

```

set.seed(42)

simulacion <- replicate(3000, {
  pesos <- normalizar_pesos(runif(length(activos)))
  metricas <- calcular_metricas(pesos, retornos_medios, covarianzas)

  c(
    retorno = metricas$retorno,
    volatilidad = metricas$volatilidad,
    sharpe = metricas$sharpe
  )
})

simulacion <- as.data.frame(t(simulacion))

```

Cada fila representa un portafolio aleatorio. La simulación es contexto visual: el punto rojo representa la selección del usuario. set.seed hace reproducible la nube de puntos.

7. Construir la interfaz

Para no repetir cinco veces el mismo slider:

```
crear_slider <- function(activo, valor) {  
  sliderInput(  
    inputId = paste0("peso_", tolower(activo)),  
    label = paste0("Peso de ", activo, " (%)" ),  
    min = 0,  
    max = 100,  
    value = valor,  
    step = 1  
  )  
}
```

Uso:

```
sidebarPanel(  
  h4("Pesos del portafolio"),  
  crear_slider("SPY", 40),  
  crear_slider("EFA", 20),  
  crear_slider("IJS", 10),  
  crear_slider("EEM", 10),  
  crear_slider("AGG", 20)  
)
```

El inputId debe ser único. La interfaz también reserva:

```
textOutput("pesos_finales")  
tableOutput("metricas")  
plotOutput("grafico_riesgo_retorno", height = "500px")
```

8. Leer inputs con reactive

```
pesos_reactivos <- reactive({  
  pesos <- c(  
    input$peso_spy,  
    input$peso_efa,  
    input$peso_ijs,  
    input$peso_eem,  
    input$peso_agg  
  )  
  
  normalizar_pesos(pesos)  
})
```

Reactive crea una expresión que se recalcula cuando cambia alguno de sus inputs. Se consulta con paréntesis:

```
pesos_reactivos()
```

El flujo es:

```
sliderInput → input$peso_spy → pesos_reactivos() → outputs
```

9. Encadenar cálculos

```
metricas_reactivas <- reactive({  
  calcular_metricas(  
    pesos = pesos_reactivos(),  
    retornos_medios = retornos_medios,  
    covarianzas = covarianzas  
  )  
})
```

La cadena queda:

```
inputs → pesos_reactivos() → metricas_reactivas() → tabla y gráfico
```

Esta separación evita copiar la fórmula financiera dentro de cada salida.

10. Mostrar texto y tabla

```
output$pesos_finales <- renderText({  
  pesos <- round(pesos_reactivos() * 100, 1)  
  nombres <- paste(activos, pesos, "%", sep = " ")  
  paste("Pesos normalizados:", paste(nombres, collapse = " | "))  
})
```

```
output$metricas <- renderTable({  
  metricas <- metricas_reactivas()  
  
  data.frame(  
    Metrica = c("Retorno anual", "Volatilidad anual", "Ratio de Sharpe"),  
    Valor = c(  
      paste0(round(metricas$retorno * 100, 2), "%"),  
      paste0(round(metricas$volatilidad * 100, 2), "%"),  
      round(metricas$sharpe, 3)  
    )  
  )  
})
```

El identificador debe coincidir en el servidor y la interfaz:

```
output$metricas <- renderTable({ ... })
tableOutput("metricas")
```

11. Crear el gráfico reactivo

```
output$grafico_riesgo_retorno <- renderPlot({
  elegido <- metricas_reactivas()

  ggplot(simulacion, aes(x = volatilidad, y = retorno, color = sharpe)) +
    geom_point(alpha = 0.35, size = 1.5) +
    geom_point(
      data = data.frame(
        volatilidad = elegido$volatilidad,
        retorno = elegido$retorno
      ),
      aes(x = volatilidad, y = retorno),
      color = "red",
      size = 5
    ) +
    theme_minimal()
})
```

El primer grupo de puntos es la simulación. El punto rojo es el portafolio elegido. Al estar dentro de renderPlot, se actualiza con los sliders.

12. Responder a botones

```
actionButton("conservador", "Conservador")
actionButton("balanceado", "Balanceado")
actionButton("agresivo", "Agresivo")

observeEvent(input$conservador, {
  updateSliderInput(session, "peso_spy", value = 15)
  updateSliderInput(session, "peso_efa", value = 10)
  updateSliderInput(session, "peso_ijs", value = 5)
  updateSliderInput(session, "peso_eem", value = 5)
  updateSliderInput(session, "peso_agg", value = 65)
})
```

ObserveEvent escucha una acción. Aquí usa session para cambiar los controles visibles.

13. Reconstruir el flujo completo

1. Se cargan datos históricos.
2. Se calculan medias y covarianzas.
3. Se simulan portafolios para el gráfico.
4. El usuario modifica los sliders.
5. `reactive()` normaliza los pesos.
6. Las funciones calculan las métricas.
7. Los outputs actualizan la pantalla.
8. `observeEvent()` carga estrategias predefinidas.

Parte	Responsabilidad
Datos	Leer y preparar retornos
Funciones	Implementar la lógica financiera
UI	Definir controles y salidas
Reactividad	Conectar acciones y cálculos
Outputs	Mostrar resultados

14. Preguntas y ejercicio

1. ¿Qué variables son inputs?
2. ¿Qué variables son outputs?
3. ¿Qué código se ejecuta una sola vez al iniciar?
4. ¿Qué código se repite al mover un slider?
5. ¿Por qué simulacion no necesita ser reactiva?
6. ¿Dónde se anulan los retornos?
7. ¿Qué pasaría si eliminamos `normalizar_pesos`?

Agrega una salida que muestre el activo con mayor peso:

```
output$activo_principal <- renderText({
  pesos <- pesos_reactivos()
  activo <- activos[which.max(pesos)]
  paste("El activo con mayor peso es", activo)
})
```

Y agrega su espacio en la interfaz:

```
textOutput("activo_principal")
```

Cierre

La aplicación conecta finanzas, programación y desarrollo de aplicaciones: retorno, riesgo, matrices, funciones, inputs, outputs y reactividad.

Esta estructura permite extender la app con Markowitz, restricciones, nuevos datos o un benchmark sin perder claridad.